UD 1. Introducción e instalación.

1. Introducción

Angular es un framework para el desarrollo de aplicaciones web en la parte del cliente. Está basado en el popular AngularJS (versiones 1.x), pero ha cambiado mucho con respecto a su antecesor. Dos mejoras significativas son: un rendimiento muy superior, y una API simplificada con menos conceptos que aprender.

Angular esta desarrollado en TypeScript, y es el lenguaje recomendado para construir aplicaciones con este framework. Este lenguaje es básicamente JavaScript con algunas características adicionales y el tipado de variables y funciones. Esto mejora mucho la depuración de aplicaciones en tiempo de desarrollo, el autocompletado por parte del editor (intellisense), etc. Al pasar por un proceso de compilación a JavaScript, el resultado será código compatible con todos los navegadores actuales.

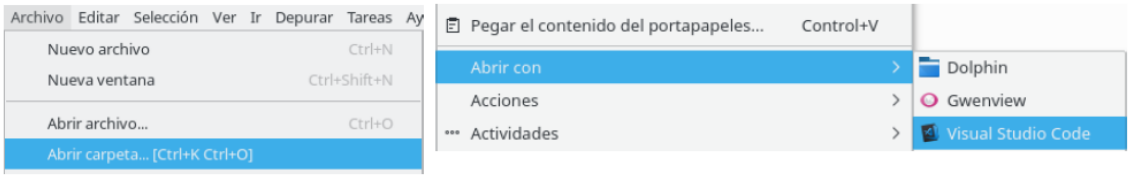
Algunas de las principales características de Angular son:

- Introduce expresividad en el código HTML a través de la interpolación de variables, data-binding, directivas, etc.
- Tiene un diseño modular. Solo se importan las características que necesitamos en la aplicación (mejora de tamaño y rendimiento). Además, permite separar nuestra aplicación en módulos de forma que el navegador vaya cargando en cada momento lo que necesita (lazy loading) → Mejora de tiempos de carga.
- Es fácil crear componentes reutilizables para la aplicación actual u otras.
- La integración con un servidor de backend basado en servicios web es muy sencilla.
- Permite ejecutar Angular en el lado del servidor para generar contenido estático que puedan indexar los motores de búsqueda (incapaces de ejecutar ellos Angular en el cliente). Esta característica se llama Angular Universal o Server Side Rendering.
- Potentes herramientas de desarrollo y depuración: TypeScript, Angular
- DevTools (plugin de Chrome), frameworks de pruebas como Karma o Jasmine, etc.
- Integración con frameworks de diseño como Bootstrap, Angular Material, Ionic.
- En Angular, creamos aplicaciones de una página (SPA), donde la pagina
- principal se carga entera solo una vez → Mejor rendimiento.
- Debido a su naturaleza de framework completo y modular, es la solución más efectiva para desarrollar grandes aplicaciones.

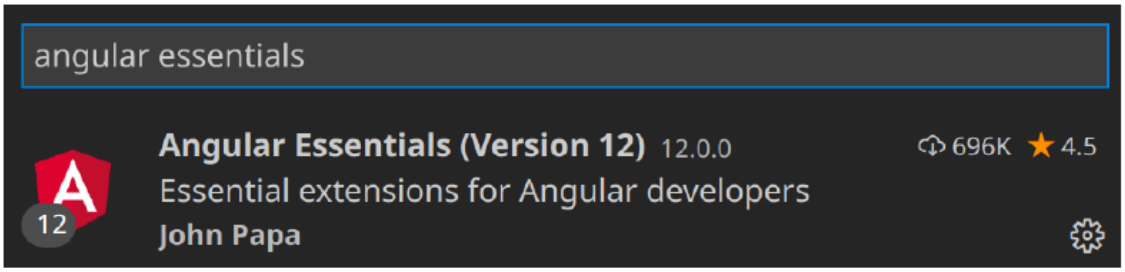
1.1. Creación de un proyecto Angular

Aunque varios editores (Atom, Webstorm, etc.) se integran bien con Angular por medio de extensiones, en este curso recomiendo utilizar Visual Studio Code. Es un editor/IDE de codigo abierto que se integra muy bien con este framework, y es el que utilizare durante el curso para ejemplos, resolver dudas, etc. Cabe destacar que este editor es una aplicacion de escritorio completamente desarrollada mediante tecnologias web (HTML, JavaScript, ···) gracias a Electron.

En VS Code un proyecto es un directorio o carpeta del sistema operativo. No existe el concepto de creacion de un proyecto como en otros IDE. Las carpetas se pueden abrir desde el propio editor, arrastrandola, mediante el comando **"code ."**, o mediante clic derecho en la carpeta → Abrir con Visual Studio Code.



El ultimo icono de la barra de la izquierda del editor, representa las extensiones que se pueden instalar para ampliar las funcionalidades (integracion con lenguajes, frameworks, FTP, GIT, ···). Para este curso os recomiendo buscar e instalar el paquete de extensiones **Angular Essentials**.



1.2. Creando un proyecto con Angular CLI

Angular CLI es la herramienta oficial que facilita la creación, configuración y gestión de proyectos Angular. Su objetivo es estandarizar la estructura de las aplicaciones, automatizar tareas repetitivas y garantizar buenas prácticas desde el primer minuto.

Con Angular 20, la CLI continúa siendo el punto de entrada recomendado para cualquier proyecto profesional o educativo.

Para instalar (o actualizar) Angular CLI, primero debemos tener instalada la herramienta Node Package Manager (NPM), que viene incluida con NodeJS (Instalad la version LTS). Una vez cumplido esto, ejecutamos (como administrador, en Linux con **sudo** delante):

```
C:\angular>npm install -g @angular/cli@20
```

Una vez instalada, puedes comprobar la versión:

```
C:\angular>ng version
```

```
Angular CLI: 20.3.15
Node: 22.19.0
Package Manager: npm 11.6.4
OS: win32 x64
```

Para generar un proyecto desde cero:

```
D:\DWEC\ANGULAR>ng new productos
```

Si quieres que tu nuevo proyecto incluya **Angular Router**, el sistema de navegación que permite tener varias páginas o vistas dentro de la aplicación.

```
? Would you like to add Angular routing? (y/N)
```

Selección del preprocesador de estilos, opciones habituales:

- CSS
- SCSS (recomendado en proyectos modernos)
- SASS
- LESS

La elección afecta a todos los estilos del proyecto.

```
? Which stylesheet format would you like to use?
> CSS              [
https://developer.mozilla.org/docs/Web/CSS
                 ]
  Sass (SCSS)      [
https://sass-lang.com/documentation/syntax#scss
                 ]
  Sass (Indented) [ https://sass-lang.com/documen

↑↓ navigate • ↵ select
```

Para iniciar el servidor de desarrollo:

```
C:\DWEC\ANGULAR>cd aplicacion

C:\DWEC\ANGULAR\aplicacion>ng serve
```

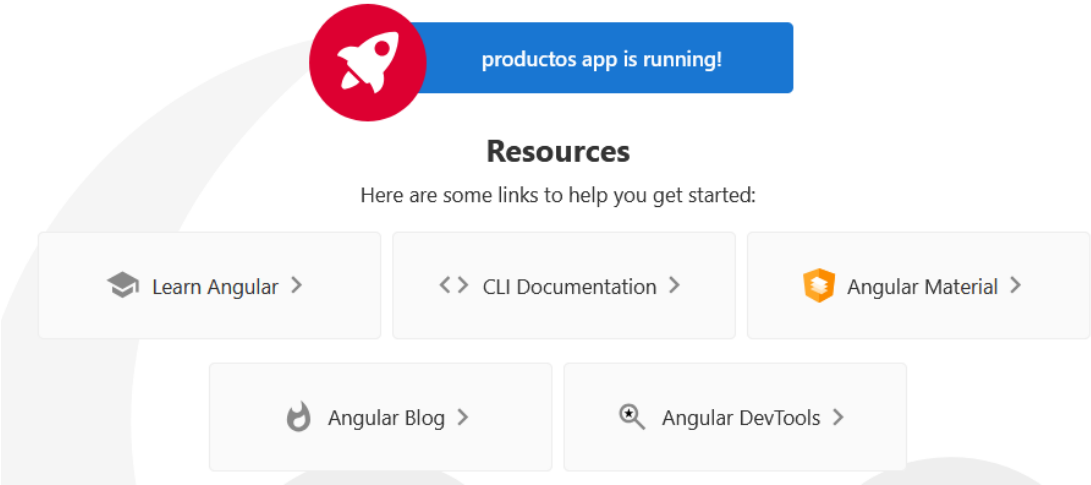
O abrir automáticamente el navegador:

```
C:\DWEC\ANGULAR>cd aplicacion

C:\DWEC\ANGULAR\aplicacion>ng serve --open
Browser bundles
```

Preguntara si quieres **enviar datos anónimos de uso** para ayudar a mejorar Angular. ¿Qué significa esta opción? - Si respondes **y**: Angular enviará *datos no identificables personalmente* sobre cómo usas el CLI (versiones, comandos ejecutados, errores comunes…). - Si respondes **N**: No se enviará nada.

```
D:\DWEC\ANGULAR\productos>ng serve --open
? Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.io/analytics. No
Global setting: enabled
Local setting: disabled
Effective status: disabled
√ Browser application bundle generation complete.

Initial Chunk Files   | Names         |  Raw Size
vendor.js             | vendor        |   2.28 MB |
polyfills.js          | polyfills     | 252.54 kB |
styles.css, styles.js | styles        | 149.82 kB |
main.js               | main          |  48.94 kB |
runtime.js            | runtime       |   6.50 kB |

                      | Initial Total |   2.72 MB

Build at: 2026-02-12T10:45:00.766Z - Hash: 62070aa02db4b93e - Time: 8591ms

** Angular Live Development Server is listening on localhost:4200, open your browser on http://localhost:4200/ **
```

Nos mostrará la siguiente pantalla



1.3. Cómo funciona Angular CLI

Angular CLI es el motor que impulsa el flujo de trabajo moderno en Angular. Su función principal es automatizar tareas complejas, garantizar buenas prácticas y proporcionar una estructura uniforme para cualquier proyecto. Con Angular 20, la CLI sigue siendo una herramienta esencial tanto para desarrolladores profesionales como para estudiantes que se inician en el framework.

Angular CLI se basa en tres principios:

- Estandarización: todos los proyectos creados con la CLI comparten la misma estructura y configuración.
- Automatización: genera código, compila, ejecuta tests y optimiza la aplicación sin intervención manual.
- Productividad: permite centrarse en la lógica de la aplicación en lugar de en la configuración del entorno.

Angular CLI funciona como una capa que coordina varias herramientas:

- Vite como sistema de compilación y servidor de desarrollo (desde Angular 17).
- TypeScript para la transpilación del código.
- Esbuild para optimizaciones de producción.
- Schematics para generar código (componentes, módulos, servicios…).
- Builders para ejecutar tareas como *serve, build, test* o *lint*.

La CLI actúa como un orquestador, ejecutando estas herramientas mediante comandos simples.

Los comandos principales de Angular CLI son:

COMANDO	DESCRIPCIÓN
ng new nombre-del-proyecto	Crear un proyecto
ng serve	Ejecutar el servidor de desarrollo
ng generate component nombre ng generate service usuarios ng generate module admin	Generar código (schematics)
ng build --configuration production	Compilar para producción
ng update	Actualizar Angular
ng test	Ejecutar pruebas

La configuración central la soporta el fichero *angular.json*, este archivo controla:

- Entradas y salidas del proyecto
- Configuraciones de compilación
- Entornos
- Builders utilizados
- Arquitectura del workspace

Angular CLI lo gestiona automáticamente, evitando configuraciones manuales complejas.

1.4. Analizando la estructura del proyecto

Cuando creas un nuevo proyecto con Angular CLI, se genera automáticamente una estructura organizada y estandarizada. Esta estructura es fundamental para comprender cómo funciona una aplicación Angular, dónde se ubican los distintos elementos y cómo se relacionan entre sí.

Con Angular 20, esta arquitectura sigue siendo modular, escalable y fácil de mantener.

Angular CLI genera una estructura similar a esta:

```
nombre-del-proyecto/
├── node_modules/
├── src/
│   ├── app/
│   ├── assets/
│   ├── environments/
│   ├── main.ts
│   ├── index.html
│   └── styles.scss
├── angular.json
├── package.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.spec.json
└── README.md
```

CARPETA / FICHERO	DESCRIPCIÓN
src/app/	Es la carpeta principal donde se desarrolla la aplicación. Contiene: **app.component.ts** → Lógica del componente raíz **app.component.html** → Plantilla del componente raíz **app.component.scss** → Estilos del componente raíz **app.module.ts** → Módulo principal de la aplicación Otros componentes, módulos, servicios, pipes, etc. Conforme el proyecto crece, esta carpeta se organiza en módulos funcionales.
src/assets/	Carpeta para recursos estáticos: Imágenes Iconos Archivos JSON Fuentes Todo lo que se coloque aquí se copia tal cual al build final
src/environments/	Contiene configuraciones por entorno: **environment.ts** → entorno de desarrollo **environment.prod.ts** → entorno de producción Permite cambiar URLs, claves o configuraciones según el entorno
src/main.ts	Es el punto de entrada de la aplicación.
src/index.html	El archivo HTML principal. Contiene: El <app-root> donde Angular inyecta la aplicación

	Metadatos Carga inicial de scripts
src/styles.scss	Archivo global de estilos. Aquí puedes definir: Variables SCSS Estilos globales Importación de librerías CSS
angular.json	El archivo más importante de configuración. Define: Cómo se compila la aplicación Configuraciones de desarrollo y producción Rutas de entrada y salida Builders utilizados Configuración de assets, estilos y scripts Angular CLI lo gestiona automáticamente.
package.json	Contiene: Dependencias del proyecto Scripts de npm Información del proyecto
tsconfig.json	Configuración global de TypeScript: Target de compilación Paths Strict mode Decoradores Angular usa varios tsconfig específicos para app, tests, etc
readme.md	Contiene todas las dependencias instaladas. No se modifica manualmente.